FinSecure Pay: Automating a Cloud-Native DevOps Pipeline

Capstone - DevOps, Cloud and AI | Individual submission

- Business problem: small fintech needs a fast, repeatable, secure way to ship a transaction-validation API with minimal manual work.
- My role: DevOps engineer - source to production-style Kubernetes deployment, with security, monitoring, alerting and recovery.
- Solution: Git/GitHub -> Jenkins -> Maven/JUnit -> SpotBugs + Trivy gates -> Docker -> Kubernetes -> Prometheus/ Grafana/Loki -> email alerts.
- Result: every change is built, tested, scanned, deployed, observed and can be rolled back from one pipeline.
- GitHub: https://github.com/badri35/capstone
- Docker Hub: https://hub.docker.com/r/badri3590/finsecure-pay | AWS EKS cluster: finsecure (us-east-1) | Jenkins, Grafana, Mailpit: local (screenshots in deck)

Task 1 - Application and Version Control

Spring Boot 4 / Java 17 / Maven / JUnit 5

- GET /health -> {status: UP, version}
- POST /validate -> 200 APPROVED | 422 REJECTED + all failed rules | 400 malformed JSON
- Rules: id format, account format (8-20 A-Z0-9) and source != destination, 0 < amount <= 10,000.00 with <= 2 decimals, currency in USD/EUR/GBP/INR/SGD
- 19 tests: 14 unit (rules, boundaries, multi-error) + 5 MockMvc (HTTP contract, Prometheus endpoint)
- Business metric finsecure_transactions_total{result}; JSON logs to stdout

```
$ git log --oneline
301b959 docs: EKS deployment evidence, screenshots and refreshed deck
6b4a542 feat: multi-arch image publish, EKS deploy target, EKS cluster config, more evidence screenshots
27e1f0e evidence: image published to Docker Hub by pipeline (build 13); refresh deck
457de4b chore: publish image under Docker Hub namespace badri3590
2f884f1 docs: README, design doc, monitoring/logging fixes, evidence and capstone deck
5c89eb3 Revert "test: introduce controlled failing test (pipeline must block deployment)"
60db28c test: introduce controlled failing test (pipeline must block deployment)
a47c498 security: migrate to Spring Boot 4.1.2 (Spring Framework 7) to clear remaining HIGH/CRITICAL CVEs
1e536db security: upgrade Spring Boot 3.3.5 -> 3.5.16 to remediate HIGH/CRITICAL CVEs

$ mvn clean verify
Tests run: 19, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Tasks 2 & 3 - Containerisation and Kubernetes on AWS EKS

Multi-arch non-root image on Docker Hub; EKS (us-east-1, 2 nodes); 2 replicas; probes; ConfigMap; LoadBalancer

- Dockerfile: multi-stage, non-root, HEALTHCHECK; pipeline publishes one amd64+arm64 image: hub.docker.com/r/badri3590/finsecure-pay
- Deployment: 2 replicas, RollingUpdate (maxUnavailable 0), readiness + liveness probes, resource limits, PDB; ConfigMap (LOG_LEVEL, APP_ENV)
- Deployed by the Jenkins pipeline (TARGET=cloud) to EKS; Service type LoadBalancer -> public AWS ELB; /health and /validate reached over the internet
- Recovery: deleted a pod -> replacement Ready in ~30 s, service stayed up
- Monitoring stack and local rollout/rollback demos ran on minikube with the same manifests

Task 3 (AWS EKS) - Deployment (2 replicas), Service (LoadBalancer), ConfigMap, pods Ready

```
$ kubectl -n finsecure get deploy,pods,svc,cm -o wide
$ kubectl -n finsecure describe deploy finsecure-pay | grep ...

NAME                          READY   UP-TO-DATE   AVAILABLE   AGE   CONTAINERS      IMAGES                                      SELECTOR
deployment.apps/finsecure-pay   2/2     2            2           70s   finsecure-pay   docker.io/badri3590/finsecure-pay:1.0.15   app=finsecure-pay

NAME                               READY   STATUS    RESTARTS   AGE   IP              NODE                              NOMINATED NODE   READINESS GATES
pod/finsecure-pay-5d75c86d-9hczp   1/1     Running   0          70s   192.168.26.141   ip-192-168-25-217.ec2.internal   <none>           <none>
pod/finsecure-pay-5d75c86d-gb4cq   1/1     Running   0          70s   192.168.52.113   ip-192-168-44-80.ec2.internal    <none>           <none>

NAME                  TYPE           CLUSTER-IP      EXTERNAL-IP                                                              PORT(S)        AGE   SELECTOR
service/finsecure-pay   LoadBalancer   10.100.54.118   a3e565fc6a603455d97eacc8853c5357-1950740779.us-east-1.elb.amazonaws.com   80:30754/TCP   76s   app=finsecure-pay

NAME                           DATA   AGE
configmap/finsecure-pay-config   2      78s
configmap/kube-root-ca.crt       1      80s

Replicas:               2 desired | 2 updated | 2 total | 2 available | 0 unavailable
StrategyType:           RollingUpdate
RollingUpdateStrategy:  0 max unavailable, 1 max surge
    Image:      docker.io/badri3590/finsecure-pay:1.0.15
    Liveness:   http-get http://:http/actuator/health/liveness delay=30s timeout=1s period=10s successThreshold=1 failureThreshold=3
    Readiness:  http-get http://:http/actuator/health/readiness delay=10s timeout=1s period=5s successThreshold=1 failureThreshold=3
  Available      True    MinimumReplicasAvailable

{"APP_ENV":"production","LOG_LEVEL":"INFO"}
```

EKS: 2/2 pods Ready, LoadBalancer, ConfigMap, probes

Task 4 (AWS EKS) - Rollout history and deployed multi-arch image

```
$ kubectl -n finsecure rollout history deploy/finsecure-pay
$ kubectl -n finsecure get deploy -o wide
$ docker buildx imagetools inspect <image>

deployment.apps/finsecure-pay
REVISION  CHANGE-CAUSE
1         <none>

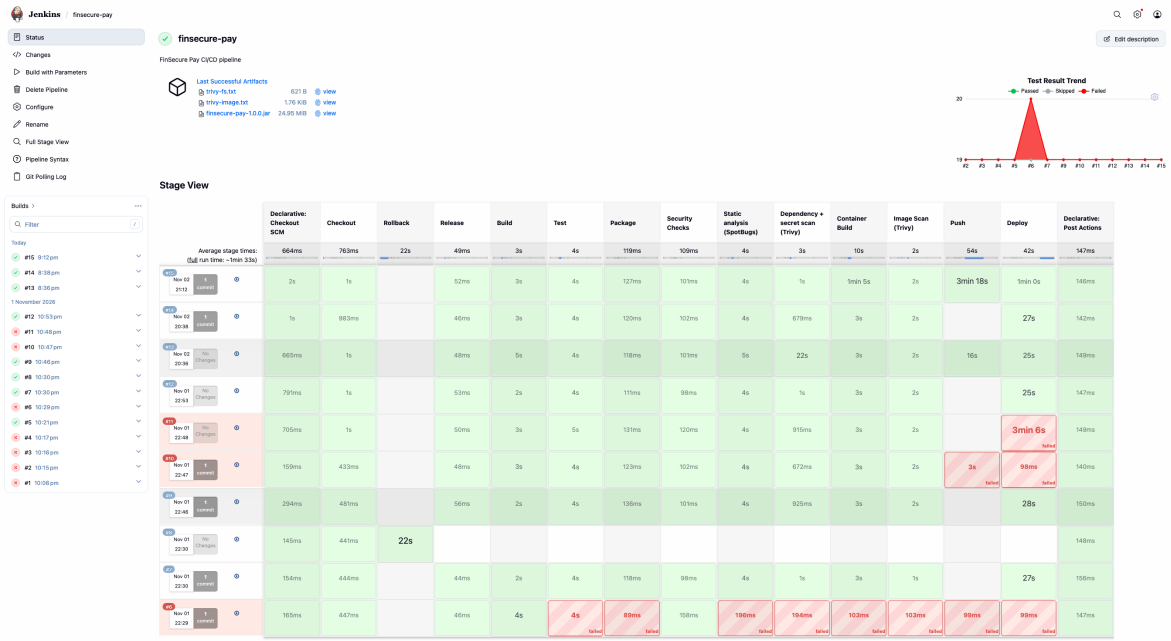

docker.io/badri3590/finsecure-pay:1.0.15

Name:      docker.io/badri3590/finsecure-pay:1.0.15
MediaType: application/vnd.oci.image.index.v1+json
  Name:        docker.io/badri3590/finsecure-pay:1.0.15@sha256:e717e818c148da1c5121c71d65ec06c68794e6f5881f26a7fed86f61ba814145
  MediaType:   application/vnd.oci.image.manifest.v1+json
  Platform:    linux/amd64
  Name:        docker.io/badri3590/finsecure-pay:1.0.15@sha256:80b19b33dd62c6b3a3cfedf3e48521575668fb98c89cfcdc2b221a3bf3f55864
  MediaType:   application/vnd.oci.image.manifest.v1+json
  Platform:    linux/arm64
```

Rollout history and multi-arch (amd64 + arm64) image deployed to EKS

Task 4 - CI/CD Pipeline (Jenkins)

Checkout > Build > Test > Package > Security > Container Build > Image Scan > Push > Deploy


Jenkins
finsecure-pay
Status
Changes
Build with Parameters
Delete Pipeline
Configure
Rename
Full Stage View
Pipeline Syntax
Git Polling Log
finsecure-pay
FinSecure Pay CI/CD pipeline
Edit description
Last Successful Artifacts
trivy-fs.txt
trivy-image.txt
finsecure-pay-1.0.0.jar
Test Result Trend
Stage View
Declarative: Checkout SCM
Checkout
Rollback
Release
Build
Test
Package
Security Checks
Static analysis (SpotBugs)
Dependency + secret scan (Trivy)
Container Build
Image Scan (Trivy)
Push
Deploy
Declarative: Post Actions
Average stage times:
664ms
763ms
22s
49ms
3s
4s
119ms
109ms
4s
3s
10s
2s
54s
42s
147ms
3min 18s
1min 0s
3min 6s
failed
failed
failed

Task 5 - DevSecOps and Security Gates

SpotBugs (static analysis), Trivy fs (deps + secrets), Trivy image; HIGH/CRITICAL blocks release

Task 5 - Trivy gate BLOCKED the release (build #3, Spring Boot 3.3.5)

```
$ grep Total / CRITICAL / skipped in build-3 console

Stage "Rollback" skipped due to when conditional
Total: 38 (HIGH: 30, CRITICAL: 8)
| org.apache.tomcat.embed:tomcat-embed-core             | CVE-2025-24813 | CRITICAL |        | 10.1.31          | 11.0.3, 10.1.35, 9.0.99   | tomcat: Potential RCE and/or information disc
losure and/o
| org.springframework:spring-webmvc                     | CVE-2026-47884 | CRITICAL |        |                  | 7.0.9                     | org.springframework/spring-webmvc: Spring Fra
mework: Remo
Stage "Container Build" skipped due to earlier failure(s)
Stage "Image Scan (Trivy)" skipped due to earlier failure(s)
Stage "Push" skipped due to earlier failure(s)
Stage "Deploy" skipped due to earlier failure(s)
Finished: FAILURE
```

Build #3: Trivy gate found 38 HIGH/CRITICAL -> Container Build/Push/Deploy skipped

- #2: SpotBugs EI_EXPOSE_REP in ValidationResult -> fixed with defensive copies
- #3: Trivy: 38 HIGH/CRITICAL (8 CRITICAL) in Spring Boot 3.3.5 -> BLOCKED
- #4: Boot 3.5.16 still 11 findings (fixes only in Spring Framework 7) -> BLOCKED
- #5: migrated to Spring Boot 4.1.2 -> 0 findings, image clean -> deployed
- Secrets: none in git; Jenkins credential store; Grafana password generated into a K8s Secret

Task 5 - After upgrade to Spring Boot 4.1.2: dependency and image scans clean

```
$ trivy fs ... (gate)
$ trivy image docker.io/badri3590/finsecure-pay:1.0.13 (gate)

Report Summary

┌──────────┬──────┬─────────────────┬─────────┐
│  Target  │ Type │ Vulnerabilities │ Secrets │
├──────────┼──────┼─────────────────┼─────────┤
│ pom.xml  │ pom  │        0        │    -    │
└──────────┴──────┴─────────────────┴─────────┘
Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)

trivy fs exit=0


┌──────────────────────────────────────────────────┬──────────┬─────────────────┐
│                      Target                      │   Type   │ Vulnerabilities │
├──────────────────────────────────────────────────┼──────────┼─────────────────┤
│ docker.io/badri3590/finsecure-pay:1.0.13 (ubuntu │  ubuntu  │        0        │
│ 26.04)                                           │          │                 │
├──────────────────────────────────────────────────┼──────────┼─────────────────┤
│ app/app.jar                                      │   jar    │        0        │
├──────────────────────────────────────────────────┼──────────┼─────────────────┤
│ usr/bin/pebble                                   │ gobinary │        0        │
└──────────────────────────────────────────────────┴──────────┴─────────────────┘
Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)

trivy image exit=0
```

After upgrading to Spring Boot 4.1.2: dependency and image scans clean

Task 5 - Secrets are not in source code

```
$ git grep -nE 'ghp_|dckr_pat|AKIA|password *=' (list)
$ kubectl -n monitoring get secret
$ Jenkins credential id only

tracked files containing token patterns:
  none

Kubernetes secrets (values never printed):
NAME            TYPE     DATA   AGE
grafana-admin   Opaque   2      22h

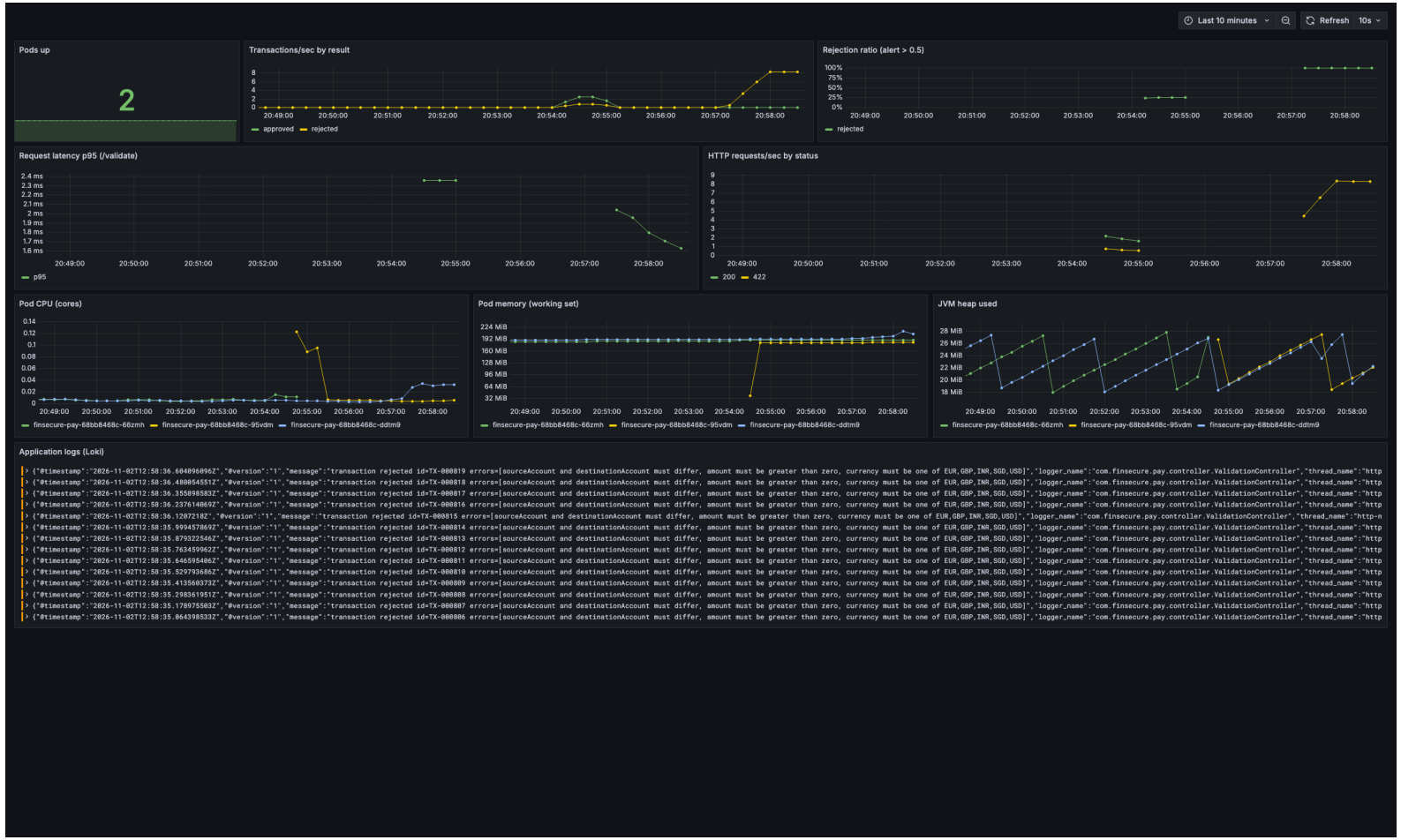
Jenkins credential store (value is AES-encrypted, never plaintext):
<id>dockerhub-credentials</id>
<username>badri3590</username>
<password>{AQAAABAAAAAw... (ciphertext)

Jenkinsfile reads it via withCredentials(...credentialsId: dockerhub-credentials):
99:                             withCredentials([usernamePassword(credentialsId: 'dockerhub-credentials',
100:                                       usernameVariable: 'REG_USER', passwordVariable: 'REG_PASS')]) {
102:                                       echo "$REG_PASS" | docker login -u "$REG_USER" --password-stdin
```

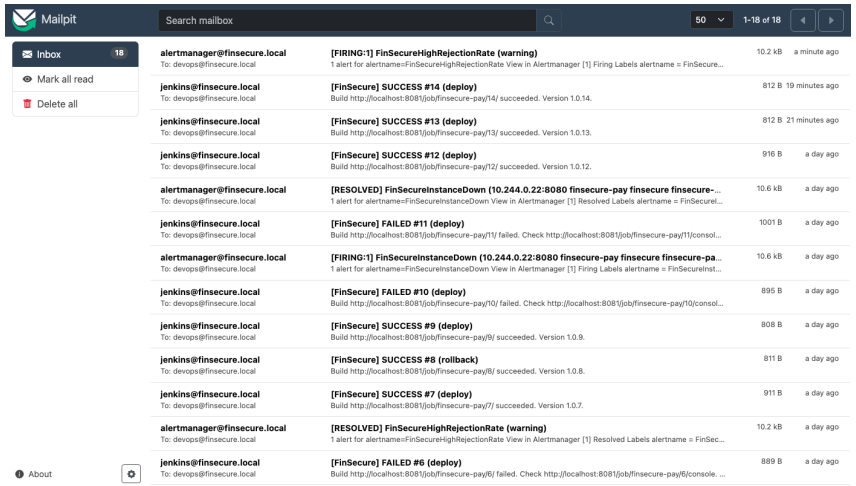
No secrets in git; credentials encrypted in Jenkins

Task 6 - Monitoring, Alerting and Logging

Prometheus + Grafana + Alertmanager + Loki; email via SMTP (Mailpit)



Live Grafana dashboard: pods, tx/s, rejection ratio, p95 latency, CPU, memory, JVM heap, logs

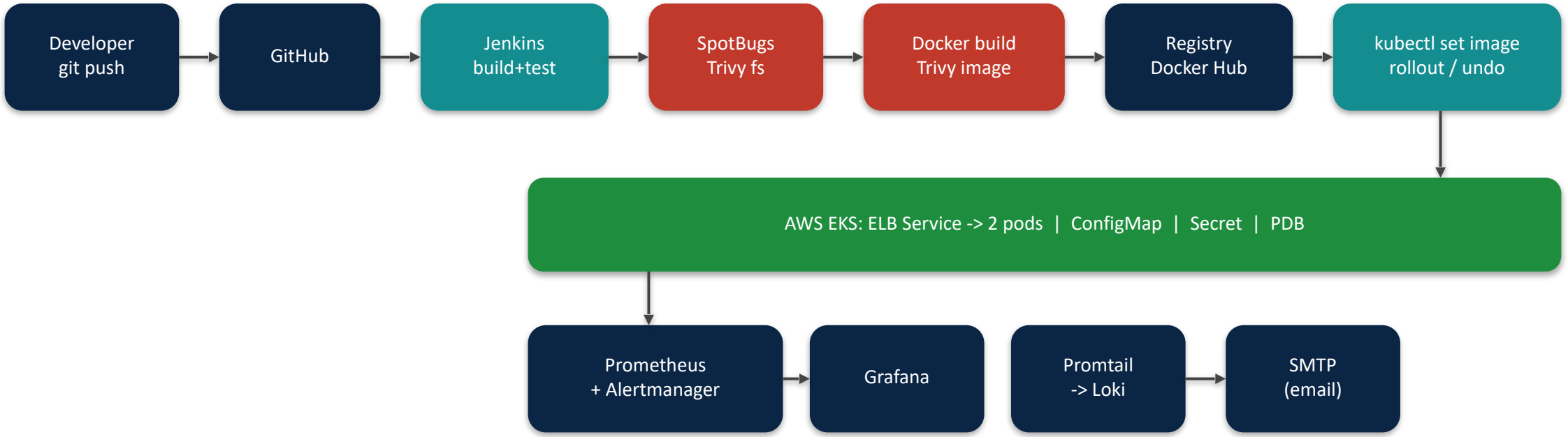


Received alert + Jenkins build emails

- Alert FinSecureHighRejectionRate (>50% for 30s) triggered with bad traffic -> FIRING -> email received
- Also: InstanceDown, HighMemory
- Logs: kubectl logs -l app=finsecure-pay, or Grafana/Loki {app="finsecure-pay"}

Task 7 - Architecture and AI Use Case

End-to-end architecture



- AI use case: LLM-assisted incident triage + anomaly detection. On a firing alert, a service collects Prometheus series, Loki logs, rollout history and Jenkins build info, masks PII, and asks an LLM for probable cause, cited evidence and a recommended action (e.g. roll back to 1.0.5).
- Why AI: correlating logs/metrics/deploys is slow and static thresholds miss unseen patterns. Benefits: lower MTTR, fewer false pages.
- Governance: advisory only (human approves rollback), cite evidence, mask account/transaction data, private model endpoint, audit prompts, deterministic fallback alerts.
- Integration: second Alertmanager webhook receiver in the monitoring namespace querying the existing Prometheus/Loki APIs.

Core Test Cases and Results

All executed in this environment; evidence in repo /evidence

Test case	Result
Run unit tests	19/19 pass (Jenkins #5, #7)
Controlled failing test	Jenkins #6 FAILED at Test; deployment not run; cluster stayed on 1.0.5
Run container, GET /health	{"status":"UP","version":"1.0.x"}
Inspect deployment	2/2 pods Running and Ready on AWS EKS (and minikube)
Deploy new version	#7 (minikube) and #15 (AWS EKS, multi-arch image) - rollout status successful
Delete a pod	Replacement created automatically on EKS and minikube, 2/2 Ready again
Critical/high finding	#3 and #4 blocked by Trivy gate; #5 clean after upgrade
Monitoring condition	Rejection-rate alert FIRING; email received in Mailpit
Rollback	#8 ACTION=rollback -> 1.0.5

Task 3 Evidence - AWS EKS: External Access and Self-Healing

Task 3 (AWS EKS) - External access through the AWS load balancer

```
$ kubectl -n finsecure get svc finsecure-pay
$ curl http://<elb-hostname>/health
$ curl -X POST http://<elb-hostname>/validate

NAME           TYPE           CLUSTER-IP      EXTERNAL-IP                                                                PORT(S)        AGE
finsecure-pay  LoadBalancer   10.100.54.118   a3e565fc6a603455d97eacc8853c5357-1950740779.us-east-1.elb.amazonaws.com   80:30754/TCP   82s

ELB: a3e565fc6a603455d97eacc8853c5357-1950740779.us-east-1.elb.amazonaws.com
{"status":"UP","version":"1.0.15"}
--- valid
{"transactionId":"TX-EKS001","status":"APPROVED","errors":[]} HTTP 200
--- invalid
{"transactionId":"TX-EKS002","status":"REJECTED","errors":["sourceAccount and destinationAccount must differ","amount must be greater than zero","currency must be one of EUR,GBP,INR,SGD,US
D"]} HTTP 422
```

Public AWS load balancer: /health UP, /validate approved (200) and rejected (422)

Task 3 (AWS EKS) - Delete a pod, Kubernetes recreates it automatically

```
$ kubectl -n finsecure get pods
$ kubectl -n finsecure delete pod <one pod>
$ kubectl -n finsecure get pods -w

NAME                           READY   STATUS    RESTARTS   AGE
finsecure-pay-5d75c86d-gb4cq   1/1     Running   0          109s
finsecure-pay-5d75c86d-pcng7   0/1     Running   0          28s

pod "finsecure-pay-5d75c86d-gb4cq" deleted from finsecure namespace

NAME                           READY   STATUS              RESTARTS   AGE
finsecure-pay-5d75c86d-gb4cq   0/1     Error               0          113s
finsecure-pay-5d75c86d-pcng7   0/1     Running             0          32s
finsecure-pay-5d75c86d-zmczh   0/1     ContainerCreating   0          2s
finsecure-pay-5d75c86d-gb4cq   0/1     Error               0          113s
finsecure-pay-5d75c86d-gb4cq   0/1     Error               0          113s
finsecure-pay-5d75c86d-zmczh   0/1     Running             0          2s
finsecure-pay-5d75c86d-pcng7   1/1     Running             0          32s
finsecure-pay-5d75c86d-zmczh   1/1     Running             0          33s

NAME                           READY   STATUS    RESTARTS   AGE
finsecure-pay-5d75c86d-pcng7   1/1     Running   0          72s
finsecure-pay-5d75c86d-zmczh   1/1     Running   0          42s

service stays reachable:
{"version":"1.0.15","status":"UP"}
```

One pod deleted -> Kubernetes creates a replacement; service stays reachable

Lessons Learned and Future Improvements

- Lesson: a real baseline scan found real CVEs - gates are only convincing on real data; the fix required a major framework upgrade (Boot 3 -> 4).
- Lesson: multi-arch matters (no arm64 JRE-alpine tag); pin bases and test on target architecture.
- Lesson: observability needs to be validated end to end (Promtail discovery failed silently; static paths fixed it).
- Next: AWS Secrets Manager via CSI driver, run the monitoring stack on EKS, GitHub webhook trigger, IAM user instead of root.
- Next: GitOps (Argo CD), HPA, cosign image signing + SBOM, OWASP ZAP DAST, Sonar quality gate, canary releases.
- Next: implement the AI triage service; add SLO-based alerts.
- Repository: https://github.com/badri35/capstone | Design doc: docs/DESIGN.md | Evidence: evidence/